

**Lab Manual for Data Structure and Algorithm
(LAB-04)
(Looping structure, Arrays, Searching in arrays, Rand() function,
Functions
and
structures)
Lab Demonstrator: Muniba Khan**

Lab 4	
Contents	Multidimensional Arrays, Multiplication of Matrix, Address-of operators, Pointer variables, Dynamic Memory Allocation, Pointer to void, Pointers and Arrays, Structures and arrays, Structure within structure, Searching and Sorting in structure

1. Objective

This lab will help you to understand and implement the concepts of multidimensional arrays, pointer operations, dynamic memory allocation, and structures (including nested structures). Students will also apply searching and sorting algorithms on structured data.

2. Lab Contents

- Multidimensional Arrays
- Matrix Multiplication
- Address-of Operators (&)
- Pointer Variables
- Dynamic Memory Allocation (new, delete)
- Pointer to Void
- Pointers and Arrays
- Structures and Arrays
- Structure within Structure
- Searching and Sorting in Structure

3.1 Multidimensional Arrays

A multidimensional array is an array of arrays. The most common type is a 2D array used to represent tables or matrices.

For example:

```
int matrix[3][3];           // 3 rows, 3 columns
```

This stores 9 integers, accessed by `matrix[i][j]`.

3.2 Matrix Multiplication

Matrix multiplication combines two matrices to produce a new matrix. It works only if the **columns of first = rows of second**.

Example:

```
C[i][j] += A[i][k] * B[k][j];
```

The resulting matrix size = [rows of A] x [columns of B].

3.3 Address-of Operator (&) and Pointer Variables

Pointers store the memory address of a variable.

```
int a = 10;
```

```
int *ptr = &a;
```

Here, `&a` gives the address of `a`, and `*ptr` accesses the value stored at that address.

3.4 Dynamic Memory Allocation

Memory can be created at runtime using new and freed using delete.

```
int *arr = new int[5]; // allocate 5 integers
delete[] arr;         // free memory
```

This is useful when the size of data is not known in advance.

3.5 Void Pointers

A void pointer can store the address of any type but needs typecasting to access the value.

```
void *ptr;
float f = 10.5;
ptr = &f;
cout << *(float*)ptr;
```

It is a generic pointer for different data types.

3.6 Structures and Arrays

Structures group different data types into one unit.

```
struct Student { int age; char name[20]; };
Student s[5]; // array of 5 students
```

Arrays of structures allow storing multiple records efficiently.

3.7 Structure within Structure

A structure can contain another structure to represent hierarchical data.

```
struct Roll { int session; int regNo; };
struct Student { char name[20]; Roll r; };
```

This is useful for modeling complex entities like student records with detailed info.

3.8 Searching and Sorting in Structure

We can **store data** (like name, age, salary, etc.) inside a **structure** and then **search or sort** it easily — just like arrays.

```
if(strcmp(emp[i].name, "Ali")==0) // Linear Search
// Bubble Sort by salary
if(emp[j].salary < emp[j+1].salary) swap(emp[j], emp[j+1]);
```

It helps organize and retrieve structured data efficiently.

3.8 Searching and Sorting in Structure

```
#include <iostream>
#include <cstring>
using namespace std;

struct Employee {
    char name[20];
    float salary;
};

int main() {
    Employee emp[3] = { {"Ali", 50000}, {"Sara", 60000}, {"Omar",
45000} };

    // Linear Search
    for (int i = 0; i < 3; i++) {
        if (strcmp(emp[i].name, "Ali") == 0)
            cout << "Found " << emp[i].name << " with salary " <<
emp[i].salary << endl;
    }

    // Bubble Sort by salary (descending)
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++) {
            if (emp[j].salary < emp[j + 1].salary)
                swap(emp[j], emp[j + 1]);
        }
    }

    cout << "\nEmployees sorted by salary:\n";
    for (int i = 0; i < 3; i++)
        cout << emp[i].name << " - " << emp[i].salary << endl;

    return 0;
}
```

4. Summary

Concept	Description
2D Arrays	Used to represent data in rows and columns, e.g., matrices.
Pointers	Store memory addresses of variables.
Dynamic Memory	Allocated at runtime using new. Must be released using delete.
Void Pointer	Can store address of any data type. Must be typecast before dereferencing.
Structure	User-defined data type that holds multiple types of variables.
Nested Structure	Structure inside another structure.
Sorting & Searching	Common data processing operations applied to structured data.

5. Examples

Example 1: Matrix Multiplication using 2D Arrays

Description:

Write a program that inputs rows and columns and performs multiplication on two matrices.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int r1, c1, r2, c2;

    cout << "Enter rows and columns of first matrix: ";
    cin >> r1 >> c1;

    cout << "Enter rows and columns of second matrix: ";
    cin >> r2 >> c2;

    // Check if multiplication is possible
    if (c1 != r2) {
        cout << "Matrix multiplication not possible!" << endl;
        return 0;
    }
    int A[10][10], B[10][10], C[10][10] = {0};

    cout << "\nEnter elements of first matrix:\n";
    for (int i = 0; i < r1; i++)
```

```

        for (int j = 0; j < c1; j++)
            cin >> A[i][j];

    cout << "\nEnter elements of second matrix:\n";
    for (int i = 0; i < r2; i++)
        for (int j = 0; j < c2; j++)
            cin >> B[i][j];

    // Matrix Multiplication
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) {
            for (int k = 0; k < c1; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    cout << "\nResultant Matrix:\n";
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++)
            cout << C[i][j] << " ";
        cout << endl;
    }

    return 0;
}

```

Output Example:

```

Enter rows and columns of first matrix: 3 2
Enter rows and columns of second matrix: 2 3

Enter elements of first matrix:
1 2
2 3
4 1

Enter elements of second matrix:
4 5 1
3 6 7

Resultant Matrix:
10 17 15
17 28 23
19 26 11

...Program finished with exit code 0
Press ENTER to exit console.

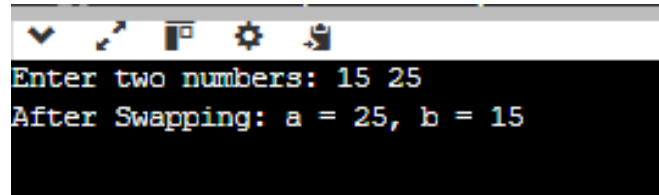
```

Task 2: Swapping Values using Pointers

Write a program that take input values from the user swaps the values with the help of pointers.

Code:

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int a, b;
6     cout << "Enter two numbers: ";
7     cin >> a >> b;
8
9     int *ptr1 = &a;
10    int *ptr2 = &b;
11
12    int temp = *ptr1;
13    *ptr1 = *ptr2;
14    *ptr2 = temp;
15
16    cout << "After Swapping: a = " << a << ", b = " << b << endl;
17    return 0;
18 }
```



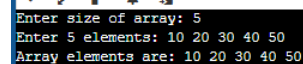
```
Enter two numbers: 15 25
After Swapping: a = 25, b = 15
```

Task 3: Dynamic Memory Allocation using Pointers

Write a program that dynamically allocate an array with the help of pointer and 'new' keyword to input elements using pointer and present on screen

Code:

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int size;
6     cout << "Enter size of array: ";
7     cin >> size;
8
9     int *arr = new int[size];
10
11    cout << "Enter " << size << " elements: ";
12    for (int i = 0; i < size; i++)
13        cin >> *(arr + i);
14
15    cout << "Array elements are: ";
16    for (int i = 0; i < size; i++)
17        cout << *(arr + i) << " ";
18
19    delete[] arr;
20    return 0;
21 }
```



```
Enter size of array: 5
Enter 5 elements: 10 20 30 40 50
Array elements are: 10 20 30 40 50
```

Output Example:

Enter size of array: 5

Enter elements: 1 2 3 4 5

Array elements are: 1 2 3 4 5

Task 4: Void Pointer Example

Write a program to initialize a void pointer, pass address of a float value to pointer and print the address of float value and the pointer

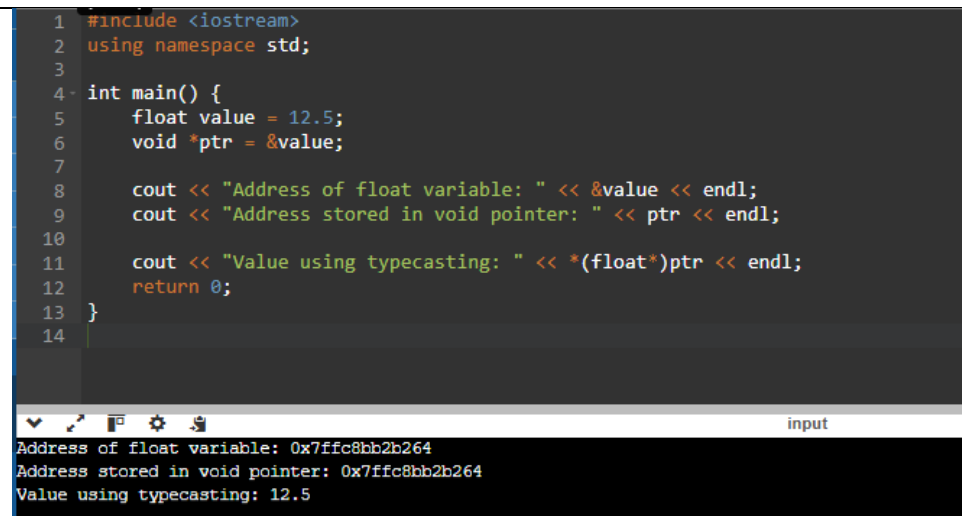
Code:

```
#include <iostream>
using namespace std;

int main() {
    float value = 12.5;
    void *ptr = &value;

    cout << "Address of float variable: " << &value << endl;
    cout << "Address stored in void pointer: " << ptr << endl;

    cout << "Value using typecasting: " << *(float*)ptr << endl;
    return 0;
}
```

A screenshot of a C++ program being executed in a terminal window. The code is displayed in a dark-themed editor with line numbers 1 through 14. The code includes the <iostream> header, uses the std namespace, and defines a main function. Inside main, a float variable 'value' is set to 12.5, and a void pointer 'ptr' is assigned its address. Three cout statements are used to print the address of 'value', the value of 'ptr', and the value of 'value' cast to a float. The output at the bottom of the terminal shows the memory address 0x7ffc8bb2b264 for both the variable address and the pointer, and the value 12.5 for the typecasted pointer.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     float value = 12.5;
6     void *ptr = &value;
7
8     cout << "Address of float variable: " << &value << endl;
9     cout << "Address stored in void pointer: " << ptr << endl;
10
11     cout << "Value using typecasting: " << *(float*)ptr << endl;
12     return 0;
13 }
14
```

Address of float variable: 0x7ffc8bb2b264
Address stored in void pointer: 0x7ffc8bb2b264
Value using typecasting: 12.5

Output Example:

Address of float variable: 0x61ff0c

Address stored in void pointer: 0x61ff0c

Value using typecasting: 12.5

Task 5: Structure within Structure

Write a C++ program to make a structure of a student consisting of integer age, char name and structure roll number that further divides into department, session, registration number and degree. Set and display all the values from main function and display the record on screen to present the access of structure within structure

Code:

```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  // Structure for roll number details
5  struct RollNo {
6      char dep[10];      // department
7      int session;
8      int regNo;         // registration number
9      char degree[10];
10 };
11
12 // Structure for student
13 struct Student {
14     int age;
15     char name[20];
16     RollNo rn;          // structure within structure
17 };
18
```

```

19 int main() {
20     Student s1;
21
22     cout << "Enter name: ";
23     cin >> s1.name;
24
25     cout << "Enter age: ";
26     cin >> s1.age;
27
28     cout << "Enter department: ";
29     cin >> s1.rn.dep;
30
31     cout << "Enter session: ";
32     cin >> s1.rn.session;
33
34     cout << "Enter registration number: ";
35     cin >> s1.rn.regNo;
36
37     cout << "Enter degree: ";
38     cin >> s1.rn.degree;
39
40     cout << "\n----- Student Record ----- \n";
41     cout << "Name: " << s1.name << endl;
42     cout << "Age: " << s1.age << endl;
43     cout << "Department: " << s1.rn.dep << endl;
44     cout << "Session: " << s1.rn.session << endl;
45     cout << "Registration No: " << s1.rn.regNo << endl;
46     cout << "Degree: " << s1.rn.degree << endl;
47
48     return 0;
49 }
50

```

Output Example:

```

main.cpp
18
19 int main() {
20     Student s1;
21
22     cout << "Enter name: ";
23     cin >> s1.name;
24
25     cout << "Enter age: ";
26     cin >> s1.age;
27
28     cout << "Enter department: ";
29     cin >> s1.rn.dep;
30
31     cout << "Enter session: ";
32     cin >> s1.rn.session;
33
34     cout << "Enter registration number: ";
35     cin >> s1.rn.regNo;
36
37     cout << "Enter degree: ";
38     cin >> s1.rn.degree;
39
40     cout << "\n----- Student Record ----- \n";
41     cout << "Name: " << s1.name << endl;
42     cout << "Age: " << s1.age << endl;
43     cout << "Department: " << s1.rn.dep << endl;
44     cout << "Session: " << s1.rn.session << endl;
45     cout << "Registration No: " << s1.rn.regNo << endl;
46     cout << "Degree: " << s1.rn.degree << endl;
47
48     return 0;
49 }
50

```

Enter name: MUNIBA
Enter age: 22
Enter department: Cyber
Enter session: 2025
Enter registration number: 829
Enter degree: BS

----- Student Record -----
Name: MUNIBA
Age: 22
Department: Cyber
Session: 2025
Registration No: 829
Degree: BS

Task 6: Sorting and Searching in Structure

Write a program using structures that first inputs the data for 3 employees including their names, emp-ids and salary. Apply bubble sort on the data entered and sort the employees on basis of highly paid salary. Also apply linear search on this data to search the employee with his name and display its further record on screen.

Code:

```
#include <iostream>
#include <cstring>
using namespace std;

struct Employee {
    char name[20];
    int empID;
    float salary;
};

int main() {
    Employee emp[3];

    cout << "Enter data for 3 employees:\n";
    for (int i = 0; i < 3; i++) {
        cout << "Name: ";
        cin >> emp[i].name;
        cout << "Emp ID: ";
        cin >> emp[i].empID;
        cout << "Salary: ";
        cin >> emp[i].salary;
    }

    // Bubble Sort by salary (descending)
    for (int i = 0; i < 3 - 1; i++) {
        for (int j = 0; j < 3 - i - 1; j++) {
            if (emp[j].salary < emp[j + 1].salary) {
                Employee temp = emp[j];
                emp[j] = emp[j + 1];
                emp[j + 1] = temp;
            }
        }
    }

    cout << "\nEmployees sorted by salary:\n";
    for (int i = 0; i < 3; i++) {
        cout << emp[i].name << " (ID: " << emp[i].empID << ") Salary: " << emp[i].salary << endl;
    }

    // Linear Search by name
    char searchName[20];
    cout << "\nEnter name to search: ";
    cin >> searchName;

    bool found = false;
    for (int i = 0; i < 3; i++) {
        if (strcmp(emp[i].name, searchName) == 0) {
            cout << "Record Found:\n";
            cout << "Name: " << emp[i].name << "\nID: " << emp[i].empID << "\nSalary: " << emp[i].salary << endl;
            found = true;
            break;
        }
    }

    if (!found)
        cout << "Employee not found." << endl;

    return 0;
}
```

Output Example:

Employees sorted by salary:

Ali (ID: 101) Salary: 75000

Ahmed (ID: 102) Salary: 60000

Bilal (ID: 103) Salary: 50000

Enter name to search: Ahmed

Record Found:

Name: Ahmed

ID: 102

Salary: 60000

Lab Task**Task 7: Mini Student Record System (Final Lab Task)**

Objective: Integrate all concepts.

Assignment:

- Create an array of 5 students using nested structures.
- Each student has: name, age, roll (with department, session, registration, degree), and 3 subject marks.
- Compute average GPA for each student.
- Sort students by GPA (highest first).
- Search a student by registration number and display full record.

Concepts

Binary Search

Binary Search is an **efficient algorithm** to find an element in a **sorted list or array**.

Instead of checking every element one by one (like Linear Search), Binary Search divides the list into halves to find the target faster.

Steps:

1. Start with a **sorted** array.
2. Find the **middle element**.
3. Compare it with the value you want to find (called the **key**):
 - If the middle element == key → Found!
 - If the key < middle element → Search **left half**.
 - If the key > middle element → Search **right half**.
4. Repeat the process until the key is found or the list is empty.

Example:

Array:

[2, 4, 6, 8, 10, 12, 14]

Search for: 10

Steps:

- *Middle = 8 → 10 > 8 → search right half*
- *New middle = 12 → 10 < 12 → search left half*
- *New middle = 10 → Found!*

```
main.cpp
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int arr[] = {2, 4, 6, 8, 10, 12, 14};
6      int n = 7;
7      int key, low = 0, high = n - 1, mid;
8      cout << "Enter number to search: ";
9      cin >> key;
10
11     while (low <= high) {
12         mid = (low + high) / 2;
13
14         if (arr[mid] == key) {
15             cout << "Found at index " << mid << endl;
16             return 0;
17         }
18         else if (key < arr[mid])
19             high = mid - 1;
20         else
21             low = mid + 1;
22     }
23
24     cout << "Not found!" << endl;
25     return 0;
26 }
27
```

Bubble Sort Concept

Definition:

Bubble Sort is a simple sorting algorithm that repeatedly compares adjacent elements of an array and swaps them if they are in the wrong order.

After each pass, the largest (or smallest) element “bubbles” to its correct position.

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int arr[] = {5, 3, 8, 4, 2};
6      int n = 5;
7
8      // Bubble Sort in ascending order
9      for (int i = 0; i < n - 1; i++) {
10         for (int j = 0; j < n - i - 1; j++) {
11             if (arr[j] > arr[j + 1]) {
12                 swap(arr[j], arr[j + 1]);
13             }
14         }
15     }
16
17     cout << "Sorted array: ";
18     for (int i = 0; i < n; i++)
19         cout << arr[i] << " ";
20     return 0;
21 }
22
```

Sorted array: 2 3 4 5 8

Program finished with exit code 0

END